

# Unidad 12: Playground Avanzando

## Introducción a CRUD

### ¿Qué es CRUD?

CRUD es un acrónimo que representa las cuatro operaciones básicas que se pueden realizar en una base de datos: Crear, Leer, Actualizar y Borrar. Estas operaciones son fundamentales para la gestión de datos y son esenciales en el desarrollo de aplicaciones web y de software en general.

### Definición de CRUD

- **Crear (Create):** Agregar nuevos registros a la base de datos.
  - **Leer (Read):** Recuperar y mostrar los registros existentes en la base de datos.
  - **Actualizar (Update):** Modificar los registros existentes en la base de datos.
  - **Borrar (Delete):** Eliminar los registros existentes de la base de datos.
- 

### Importancia de CRUD en la Gestión de Datos

CRUD es importante porque proporciona una forma estructurada de interactuar con la base de datos. Permite a los desarrolladores realizar operaciones esenciales de manera consistente y predecible, facilitando la gestión y manipulación de datos en aplicaciones.

---

### CRUD en el Desarrollo Web

En el contexto del desarrollo web, CRUD se implementa a través de vistas y formularios que permiten a los usuarios interactuar con la base de datos. Por ejemplo:

- **Crear:** Formularios de registro o entrada de datos.
- **Leer:** Páginas de visualización de datos o resultados de búsqueda.
- **Actualizar:** Formularios de edición de registros existentes.
- **Borrar:** Botones o enlaces que permiten eliminar registros.

### Implementación de CRUD

En Django, implementar CRUD es un proceso estructurado que utiliza modelos, vistas y plantillas para manejar las operaciones básicas de la base de datos. A continuación, se

describe cómo empezar a trabajar con CRUD en Django, incluyendo la creación de datos en el panel de administración y la modificación de la representación de los objetos en la base de datos.

---

## Creación de Datos en el Panel de Administración

Django proporciona un panel de administración que facilita la creación, lectura, actualización y eliminación de datos. Para usar el panel de administración, primero necesitas registrar tus modelos en el archivo `admin.py` de tu aplicación.

## Registro de Modelos

Supongamos que tienes una aplicación `AppCoder` con un modelo `Estudiante`.

`AppCoder/admin.py`:

```
'''python

from django.contrib import admin
from .models import Estudiante

admin.site.register(Estudiante)
'''
```

Con este registro, puedes acceder al panel de administración y gestionar los datos de `Estudiante`.

---

## Acceso al Panel de Administración

**1. Crear un superusuario:** Si no lo has hecho ya, crea un superusuario con el siguiente comando:

```
'''bash

python manage.py createsuperuser
'''
```

**2. Iniciar el servidor de desarrollo:**

```
'''bash

python manage.py runserver
```

```
'''
```

**3. Acceder al panel de administración:** Ve a `http://127.0.0.1:8000/admin/` e inicia sesión con las credenciales del superusuario.

## Modificación de la Representación de los Objetos

Para mejorar la legibilidad de los registros en el panel de administración y en otras partes de la aplicación, puedes modificar la representación de los objetos utilizando el método `__str__` en tus modelos.

---

## Ejemplo de Método `__str__`

`AppCoder/models.py`:

```
'''python

from django.db import models

class Estudiante(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()

    def __str__(self):
        return f'{self.nombre} {self.apellido}'
'''
```

Con este método, cada instancia del modelo `Estudiante` se representará de manera más clara y legible.

---

## Ejemplo de Formulario de estudiantes

```
'''python

from django import forms

class EstudianteForm(forms.Form):
    nombre = forms.CharField()
    apellido = forms.CharField()
```

```
email = forms.EmailField()

def save(self, commit=True):
    # Crear una instancia del modelo Estudiante con los datos del formulario
    estudiante = Estudiante(
        nombre=self.cleaned_data['nombre'],
        apellido=self.cleaned_data['apellido'],
        email=self.cleaned_data['email']
    )
    if commit:
        estudiante.save()
    return estudiante
...
```

---

## Implementación de CRUD en Django

### Crear

Para crear un nuevo registro, puedes usar el panel de administración o crear vistas y formularios personalizados.

#### Ejemplo de vista para crear un estudiante:

```
```python

from django.shortcuts import render, redirect
from .models import Estudiante
from .forms import EstudianteForm

def crear_estudiante(request):
    if request.method == 'POST':
        form = EstudianteForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('lista_estudiantes')
    else:
        form = EstudianteForm()
    return render(request, 'AppCoder/crear_estudiante.html', {'form': form})
...
```
```

---

### Leer

Para leer y mostrar datos, puedes crear vistas que recuperen registros de la base de datos y plantillas que los presenten.

### Ejemplo de vista para listar estudiantes:

```
```python

from django.shortcuts import render
from .models import Estudiante

def lista_estudiantes(request):
    estudiantes = Estudiante.objects.all()
    return render(request, 'AppCoder/lista_estudiantes.html', {'estudiantes': estudiantes})
...

```

---

## Actualizar

Para actualizar un registro existente, puedes usar vistas y formularios de edición.

### Ejemplo de vista para actualizar un estudiante:

```
```python

from django.shortcuts import render, get_object_or_404, redirect
from .models import Estudiante
from .forms import EstudianteForm

def actualizar_estudiante(request, pk):
    estudiante = get_object_or_404(Estudiante, pk=pk)
    if request.method == 'POST':
        form = EstudianteForm(request.POST, instance=estudiante)
        if form.is_valid():
            form.save()
            return redirect('lista_estudiantes')
    else:
        form = EstudianteForm(instance=estudiante)
    return render(request, 'AppCoder/actualizar_estudiante.html', {'form': form})
...

```

---

## Borrar

Para borrar un registro, puedes crear vistas que manejen la eliminación de datos.

### Ejemplo de vista para borrar un estudiante:

```
```python

from django.shortcuts import get_object_or_404, redirect
from .models import Estudiante

```

```
def borrar_estudiante(request, pk):
    estudiante = get_object_or_404(Estudiante, pk=pk)
    estudiante.delete()
    return redirect('lista_estudiantes')
'''
```

---

## Conclusión

CRUD es fundamental para la gestión de datos en cualquier aplicación. En Django, puedes implementar fácilmente las operaciones CRUD utilizando vistas, formularios y el panel de administración. Este enfoque estructurado te permite interactuar de manera eficiente con la base de datos, asegurando que los datos se manejen de forma consistente y segura

---

## CRUD: Delete y update

### Update

En Django, la operación de actualización de datos (UPDATE) permite modificar los registros existentes en la base de datos. Esta operación es esencial para mantener los datos actualizados y corregir cualquier error. A continuación, se explica cómo implementar la operación de actualización en Django, incluyendo la creación de la vista, la URL y el template necesarios para actualizar registros.

### Paso 1: Crear la Vista para Actualizar Registros

La vista para actualizar registros debe obtener el objeto que se desea modificar, mostrar un formulario prellenado con los datos actuales del objeto y luego guardar los cambios realizados por el usuario.

**Ejemplo de vista para actualizar un estudiante:**

`AppCoder/views.py`:

```
'''python

from django.shortcuts import render, get_object_or_404, redirect
from .models import Estudiante
from .forms import EstudianteForm

def actualizar_estudiante(request, pk):
    estudiante = get_object_or_404(Estudiante, pk=pk)
```

```

if request.method == 'POST':
    form = EstudianteForm(request.POST, instance=estudiante)
    if form.is_valid():
        form.save()
        return redirect('lista_estudiantes')
else:
    form = EstudianteForm(instance=estudiante)
return render(request, 'AppCoder/actualizar_estudiante.html', {'form': form})
...

```

En este ejemplo:

- Se obtiene el objeto `Estudiante` correspondiente al `pk` (clave primaria) proporcionado.
- Si la solicitud es `POST`, se procesa el formulario con los datos proporcionados.
- Si el formulario es válido, se guardan los cambios y se redirige al usuario a la lista de estudiantes.
- Si la solicitud no es `POST`, se muestra el formulario prellenado con los datos del estudiante.

## Paso 2: Definir la URL para la Vista de Actualización

Se debe definir una URL que apunte a la vista de actualización. Esto permite que la aplicación acceda a la vista cuando el usuario desea actualizar un registro.

**Ejemplo de URL para actualizar un estudiante:**

**`AppCoder/urls.py`:**

```

'''python

from django.urls import path
from .views import actualizar_estudiante

urlpatterns = [
    # Otras rutas
    path('estudiante/actualizar/<int:pk>/', actualizar_estudiante,
name='actualizar_estudiante'),
]
'''

```

En este ejemplo, la URL `estudiante/actualizar/<int:pk>/` enlaza a la vista `actualizar\_estudiante`. El parámetro `pk` en la URL permite identificar el registro específico que se desea actualizar.

### Paso 3: Crear el Template para el Formulario de Actualización

El template debe contener un formulario que permita al usuario editar los datos del registro. Este formulario debe estar prellenado con los datos actuales del objeto y enviar los cambios a la vista cuando se envíe.

**Ejemplo de template para actualizar un estudiante:**

``AppCoder/templates/AppCoder/actualizar_estudiante.html``:

```
```html

<!DOCTYPE html>
<html>
<head>
  <title>Actualizar Estudiante</title>
</head>
<body>
  <h1>Actualizar Estudiante</h1>
  <form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Guardar Cambios</button>
  </form>
</body>
</html>
```
```

En este ejemplo:

- El formulario se envía utilizando el método ``POST``.
- La etiqueta ``{% csrf_token %}`` se utiliza para proteger contra ataques CSRF (Cross-Site Request Forgery).
- El formulario se renderiza con los datos actuales del estudiante utilizando ``{{ form.as_p }}``.

---

## Resumen de la Implementación de UPDATE en Django

**1. Crear la vista:** La vista ``actualizar_estudiante`` obtiene el objeto a actualizar, muestra un formulario prellenado y guarda los cambios.

**2. Definir la URL:** La URL ``estudiante/actualizar/<int:pk>/`` enlaza a la vista de actualización.



**3. Crear el template:** El template ``actualizar_estudiante.html`` contiene el formulario para editar los datos del estudiante.

Con estos pasos, se puede implementar la operación de actualización de datos (UPDATE) en Django, permitiendo modificar registros existentes de manera eficiente y segura. Esta operación es una parte fundamental del conjunto de operaciones CRUD y es esencial para mantener los datos actualizados en cualquier aplicación web.

---

# Clases Basadas en Vistas (CBV)

## Introducción

Las Clases Basadas en Vistas (Class-Based Views o CBV) en Django proporcionan una manera más estructurada y reutilizable de manejar las operaciones CRUD (Crear, Leer, Actualizar, Borrar). Las CBV permiten encapsular la lógica de las vistas en clases, simplificando la implementación y mantenimiento del código. Usar CBV para gestionar operaciones CRUD puede hacer que tu código sea más limpio, modular y fácil de extender.

## Utilidad de las CBV

- **Reutilización de Código:** Las CBV permiten heredar y reutilizar código, lo que reduce la duplicación.
  - **Organización:** Facilitan la organización de la lógica de las vistas en métodos bien definidos.
  - **Extensibilidad:** Es más sencillo extender la funcionalidad de las vistas mediante la herencia.
- 

## CBV - ListView

``ListView`` es una vista basada en clases que facilita la presentación de una lista de objetos de un modelo específico. Es útil para mostrar tablas de datos o listados de registros.

## Configuración de ListView

Para configurar una ``ListView``, debes definir ciertos atributos y métodos en la clase de vista.

### Atributos Clave:

- **``model``:** Especifica el modelo del cual se obtendrán los datos.

- **`template\_name`**: (Opcional) Especifica el template que se usará para renderizar la vista. Si no se especifica `context_object_name`, el contexto utilizará un nombre basado en el nombre del modelo en minúsculas seguido de `_list`
  - **`context\_object\_name`**: (Opcional) Nombre del contexto que se pasará al template. Si no se especifica `template_name`, Django buscará una plantilla con el nombre `{nombre_del_modelo}_list.html`
- 

## Ejemplo de ListView

**`AppCoder/views.py`**:

```
```python

from django.views.generic import ListView
from .models import Estudiante

class EstudianteListView(ListView):
    model = Estudiante
    template_name = 'AppCoder/estudiante_list.html'
    context_object_name = 'estudiantes'
...`
```

**`AppCoder/urls.py`**:

```
```python

from django.urls import path
from .views import EstudianteListView

urlpatterns = [
    path('estudiantes/', EstudianteListView.as_view(), name='estudiante_list'),
]
...`
```

**`AppCoder/templates/AppCoder/estudiante\_list.html`**:

```
```html

<!DOCTYPE html>
<html>
<head>
    <title>Lista de Estudiantes</title>
</head>
<body>
```

```
<h1>Lista de Estudiantes</h1>
<ul>
    {% for estudiante in estudiantes %}
        <li>{{ estudiante.nombre }} {{ estudiante.apellido }}</li>
    {% endfor %}
</ul>
</body>
</html>
...
```

---

## CBV - DetailView

DetailView se utiliza para mostrar los detalles de un objeto específico. Esta vista es útil para mostrar información detallada de un registro individual.

### Configuración de DetailView

**Atributos Clave:**

- **model**: Especifica el modelo del cual se obtendrán los datos.
  - **template\_name**: (Opcional) Especifica el template que se usará para renderizar la vista.
  - **context\_object\_name**: (Opcional) Nombre del contexto que se pasará al template.
- 

## Ejemplo de DetailView

**AppCoder/views.py**:

```
'''python

from django.views.generic import DetailView
from .models import Estudiante

class EstudianteDetailView(DetailView):
    model = Estudiante
    template_name = 'AppCoder/estudiante_detail.html'
    context_object_name = 'estudiante'
...'''
```

**AppCoder/urls.py**:

```
'''python
```

```
from django.urls import path
from .views import EstudianteDetailView

urlpatterns = [
    path('estudiante/<int:pk>/', EstudianteDetailView.as_view(), name='estudiante_detail'),
]
```

**`AppCoder/templates/AppCoder/estudiante\_detail.html`:**

```
'''html

<!DOCTYPE html>
<html>
<head>
    <title>Detalle del Estudiante</title>
</head>
<body>
    <h1>Detalle del Estudiante</h1>
    <p>Nombre: {{ estudiante.nombre }}</p>
    <p>Apellido: {{ estudiante.apellido }}</p>
    <p>Email: {{ estudiante.email }}</p>
</body>
</html>
'''
```

---

## CBV - UpdateView

`UpdateView` se utiliza para actualizar registros existentes en la base de datos. Esta vista muestra un formulario prellenado con los datos actuales del objeto y permite al usuario realizar cambios.

### Configuración de UpdateView

#### Atributos Clave:

- **`model`**: Especifica el modelo del cual se obtendrán los datos.
  - **`fields`**: Lista de campos del modelo que se incluirán en el formulario.
  - **`template\_name`**: (Opcional) Especifica el template que se usará para renderizar la vista.
  - **`success\_url`**: URL a la que se redirigirá tras una actualización exitosa.
-

# Ejemplo de UpdateView

**`AppCoder/views.py`:**

```
```python

from django.views.generic import UpdateView
from .models import Estudiante

class EstudianteUpdateView(UpdateView):
    model = Estudiante
    fields = ['nombre', 'apellido', 'email']
    template_name = 'AppCoder/estudiante_form.html'
    success_url = '/estudiantes/'
...

```

**\*\*`AppCoder/urls.py`\*\*:**

```
```python

from django.urls import path
from .views import EstudianteUpdateView

urlpatterns = [
    path('estudiante/editar/<int:pk>/', EstudianteUpdateView.as_view(),
    name='estudiante_update'),
]
...

```

**`AppCoder/templates/AppCoder/estudiante\_form.html`:**

```
```html

<!DOCTYPE html>
<html>
<head>
    <title>Actualizar Estudiante</title>
</head>
<body>
    <h1>Actualizar Estudiante</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Guardar Cambios</button>
    </form>
</body>
</html>

```

```
'''
```

---

## CBV - DeleteView

`DeleteView` se utiliza para eliminar registros existentes de la base de datos. Esta vista presenta una página de confirmación antes de eliminar el objeto.

### Configuración de DeleteView

#### Atributos Clave:

- `model``: Especifica el modelo del cual se obtendrán los datos.
  - `template_name``: (Opcional) Especifica el template que se usará para renderizar la vista de confirmación.
  - `success_url``: URL a la que se redirigirá tras una eliminación exitosa.
- 

## Ejemplo de DeleteView

#### `AppCoder/views.py`:

```
'''python

from django.views.generic import DeleteView
from .models import Estudiante

class EstudianteDeleteView(DeleteView):
    model = Estudiante
    template_name = 'AppCoder/estudiante_confirm_delete.html'
    success_url = '/estudiantes/'
'''
```

#### `AppCoder/urls.py`:

```
'''python

from django.urls import path
from .views import EstudianteDeleteView

urlpatterns = [
    path('estudiante/eliminar/<int:pk>', EstudianteDeleteView.as_view(),
    name='estudiante_delete'),
]
```

```
'''
```

```
`AppCoder/templates/AppCoder/estudiante_confirm_delete.html`:
```

```
'''html
```

```
<!DOCTYPE html>
<html>
<head>
  <title>Confirmar Eliminación</title>
</head>
<body>
  <h1>¿Estás seguro de que quieres eliminar este estudiante?</h1>
  <form method="post">
    {% csrf_token %}
    <button type="submit">Sí, eliminar</button>
  </form>
</body>
</html>
'''
```

---

## Conclusión

Las Clases Basadas en Vistas (CBV) en Django facilitan la implementación de operaciones CRUD de manera estructurada y eficiente. `ListView`, `DetailView`, `UpdateView` y `DeleteView` proporcionan vistas preconstruidas que reducen significativamente la cantidad de código necesario para manejar la creación, lectura, actualización y eliminación de registros. Utilizando estas CBV, puedes desarrollar aplicaciones web robustas y fáciles de mantener.

---

Claro, a continuación te presento el contenido del archivo **Clase 12.2** incorporado al **Material de Apoyo/Descargable** de la Unidad 12 en formato markdown, respetando el estilo del material original:

---

## Autenticación de Usuarios: Login, Registro y Logout

### Login - Registro - Logout

¿Por qué implementar login y logout?

Aunque ya logramos hacer CRUD con Profesores y Cursos (a mano y con `ListView`), no es adecuado que cualquier persona que acceda a la web pueda manipular los modelos.

👉 Necesitamos limitar el acceso mediante **login**, como en el panel de administración.

---

## Login

Django provee un sistema de login y logout simple, seguro y fácil de implementar.

### 1. Template básico (`login.html`):

```
<form action="" method="POST">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Inciar sesión</button>
</form>
```

### 2. Configuración en `urls.py`:

```
path('login', views.login_request, name="Login")
```

### 3. Importaciones necesarias en `views.py`:

```
from django.contrib.auth.forms import AuthenticationForm, UserCreationForm
from django.contrib.auth import login, logout, authenticate
```

### 4. Resultados esperables:

- Si los datos son correctos → acceso permitido.
- Si los datos son incorrectos → se muestra error.

---

## Registro

Permite registrar usuarios sin necesidad de usar el panel de administración.

### 1. Vista personalizada (`register`)



## 2. Configuración en `urls.py`:

```
path('register', views.register, name='Register')
```

## 3. Template para registro:

```
<form action="" method="POST">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Registrate!</button>
</form>
```

## 4. Formulario personalizado (`forms.py`):

```
class UserRegisterForm(UserCreationForm):
    email = forms.EmailField()
    password1 = forms.CharField(label="Contraseña", widget=forms.PasswordInput)
    password2 = forms.CharField(label="Repetir contraseña", widget=forms.PasswordInput)

    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']
        help_texts = {k: "" for k in fields}
```

---

# Logout

## 1. Uso de vista predefinida de Django:

```
from django.contrib.auth.views import LogoutView

path('logout', LogoutView.as_view(template_name='AppCoder/logout.html'), name='Logout')
```

## 2. Template `logout.html`:

```
<h1>Te has deslogueado...</h1>
```

---

## Accesos desde el nav

```
<a class="btn btn-primary" href="{% url 'Login' %}">Login</a>
<a class="btn btn-primary" href="{% url 'Register' %}">Registro</a>
<a class="btn btn-primary" href="{% url 'Logout' %}">Logout</a>
```

---

## Mixin y Decoradores

### Mixin (para vistas basadas en clases)

Restringe el acceso a vistas solo a usuarios autenticados:

```
from django.contrib.auth.mixins import LoginRequiredMixin
```

```
class ClaseQueNecesitaLogin(LoginRequiredMixin):
    # lógica de la clase
```

### Decoradores (para funciones **def**)

Sirven para validar la identidad de forma rápida:

```
from django.contrib.auth.decorators import login_required
```

```
@login_required
def mi_vista_restringida(request):
    ...
```

Si el usuario no está logueado, Django redireccionará a una URL predefinida.

👉 Esta URL debe establecerse en **settings.py**:

```
LOGIN_URL = '/login'
```

---

### Más información

 [Documentación con más ejemplos de mixin y decoradores en Django](#)

---

# Material complementario:

## Recursos complementarios

¿Quieres saber más?

- [Registro](#)
- [Vista](#)